

Evaluating TrackNet for High-Speed Tennis Ball Tracking: A Comparative Study of Spatio-Temporal Deep Learning Against Fundamental Computer Vision Baselines Across Broadcast and Adversarial Views

Emir Uncu
Middle East Technical University
Department of Electrical and
Electronics Engineering
e273968@metu.edu.tr

GitHub Repo:
<https://github.com/emiiruncu10/EE-584-Term-Project.git>

Youtube Link:
<https://www.youtube.com/watch?v=5mnEtxN5TVk>

Abstract— Tracking a tennis ball in video is difficult because the ball is small, moves very fast, and often appears blurred or briefly hidden. TrackNet is a deep learning network, that addresses this by reading three consecutive frames at once and predicting the ball’s position from a heatmap, allowing it to learn the ball’s motion pattern rather than just its appearance. In this work, we evaluate TrackNet Model II and compare it against four fundamental computer vision trackers from the standard curriculum: HSV color segmentation, CAMShift, Lucas-Kanade optical flow, and MOG2 background subtractor paired with a Kalman filter. We run this comparison on two datasets. The first is the clean broadcast footage used as a control, and the second is our own labeled video recorded from a fixed back-court camera, used as a harder stress test. All five trackers are evaluated on the exact same frames within each dataset, and we measure how accurately each one locates the ball using positioning error, with a 5-pixel threshold for a correct detection. On the broadcast footage, TrackNet clearly outperforms every classical method, reaching a 4.76-pixel mean error and locating the ball within 5 pixels in 97.8% of its detections, while the classical baselines stay below 63%. The picture changes on our own footage. There TrackNet’s detection precision falls to 37.9% because it was trained on a different camera viewpoint, while the MOG2 and Kalman baselines improves to 60.2% because a fixed camera makes background subtraction work well. These results show that the deep learning method is the strongest tracker overall but is sensitive to the camera viewpoint, and that a simple motion-based method can become competitive, or even better at avoiding false detections, when the recording conditions suit it. The main lesson is that no single tracker is best in all situations, and the right choice depends on the viewing geometry of the video.

I. INTRODUCTION

Tracking the ball is one of the most useful pieces of information in sports video analysis. In tennis, the ball’s trajectory is used to study player performance, review tactics, and assist referees with line calls. Professional systems such as Hawk-Eye can do this very accurately, but they rely on multiple high-end cameras placed at fixed positions and operated by dedicated staff. This setup is far too expensive

for amateurs, coaches, or small clubs. As a result, there is strong interest in extracting ball trajectories directly from ordinary broadcast video or footage recorded with a single consumer camera.

This task is harder than tracking objects in most other applications. A tennis ball is small, sometimes only a few pixels wide, and it can travel at very high speeds. The fastest recorded tennis serve is around 250 kilometers per hour, which means the ball can move a long distance between two frames. Because consumer and broadcast cameras use a relatively long shutter time, the fast-moving ball often appears blurred or stretched into a short streak, and in some frames it is briefly hidden behind a player or blends into the background. These conditions cause classical image processing methods to fail often, either by missing the ball or by locking onto the wrong object.

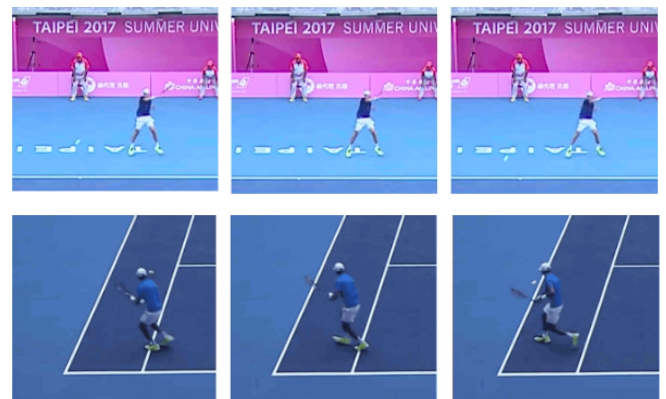


Figure 1: Examples of the main challenges in tennis ball tracking, shown as sequences of three consecutive frames from the dataset [1]. Top row: a case where the ball is hardly visible in the middle frame because its color blends into the background, yet its position can still be inferred from the neighboring frames. Bottom row: a case where the ball is occluded by the player, where again the surrounding frames help estimate its location.

To deal with these problems, Huang et al. proposed TrackNet, a deep learning network designed specifically for tracking high-speed tiny objects [1]. Instead of looking at a single image, TrackNet reads several consecutive frames at the same time. This allows it to learn not only what the ball looks like but also how it moves, so it can still estimate the ball's position when a single frame is blurred or unclear. The network is built from a VGG-16 style encoder followed by an upsampling decoder, and instead of directly outputting a coordinate, it produces a heatmap which a bright spot marks the most likely ball location. The final position is then read from this heatmap. In the original paper, this three-frame version, called Model II, reached very high precision on broadcast tennis video.

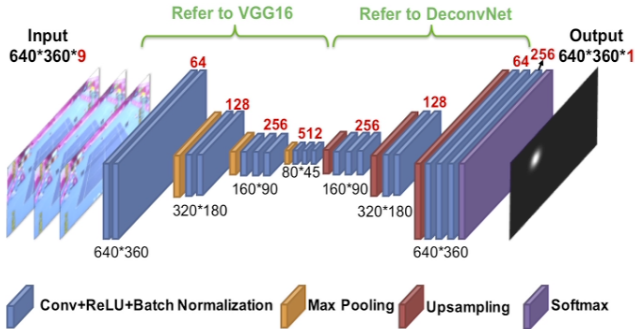


Figure 2: The TrackNet architecture diagram, redrawn from Fig.9 of the original paper [1], showing three input frames flowing through the encoder-decoder into a heatmap.

Although TrackNet performs very well in its original paper, that evaluation was done mainly on clean broadcast footage from a single tournament. This leaves an important question open: how well does TrackNet work when the videos looks different from what it was trained on, and how does it really compare against the simple, fundamental computer vision methods? Understanding this is important for two reasons. First, broadcast video is only one of many possible recording conditions; an amateur is far more likely to film a match with a single fixed camera placed behind the court. Second, comparing a complex deep learning model against basic methods shows whether its added cost is acutally justified, and where it is not. To answer these questions, we evaluate TrackNet Model II and compare it against four fundamental computer vision trackers that cover the two main classical ideas for tracking. The appearance-based methods follow the ball using it's color: HSV color segmentation, which searches each frame independently, and CAMShift, which tracks a color region over time. The motion-based methods follow the ball using how pixels move: Kanade-Lucas optical flow, which tracks feature points between frames, and a MOG2 background subtractor combined with a Kalman filter, which detects moving objects and predicts their position. We test all of these on two datasets. The first is clean broadcast footage, used as a control. The second is our own labeled footage recorded from a single fixed back-court camera, used as a harder test under different conditions.



Figure 3: Broadcast control dataset (high, wide TV angle)



Figure 4: Back-court dataset (low, fixed amateur angle)

The main contributions of this work are as follows:

- An evaluation of TrackNet Model II on clean broadcast control dataset and on our own labeled dataset recorded from a different camera viewpoint, testing how well the method generalizes beyond its original setting.
- A systematic five-way comparison between TrackNet and four fundamental computer vision trackers that cover both appearance-based and motion-based approaches, where all methods are evaluated on the exact same frames within each dataset for a fair comparison.
- A cross-view analysis showing that TrackNet is clearly the best tracker on broadcast video but becomes much less reliable on the fixed-camera footage, while the simple motion-based MOG2 and Kalman method becomes competitive, and even more reliable at avoiding false detections, when the camera is static.
- A discussion of the failure modes of each method, including cases where both the classical trackers and TrackNet itself break down, together with the reasons behind these failures.

The rest of this report is organized as follows. Section II describes the methodology, including the datasets, the tracking methods, and the evaluation protocol. Section III presents the experimental results and analysis for both datasets. Finally, Section IV concludes the report with the key findings and recommendations.

II. METHODOLOGY

This section describes the experimental setup, including the two datasets, the tracking methods we evaluate, their configuration, and the evaluation protocol used to compare them.

A. Datasets

We use two datasets that are stored and handled in different ways. The first is used as a clean control, and the second is used as a harder stress test under different recording conditions.

Dataset1 (Broadcast Control)

This dataset comes from the public TrackNet tennis dataset [1] and consists of professional broadcast footage recorded from a high, wide television angle. The frames are already extracted and saved as images, and each frame has a label file giving the ball’s visibility, its X and Y position, and its trajectory type. For our experiments we use five clips, giving a total of 815 frames. Because the frames are already on disk, our pipeline loads them directly and groups them into short sequences.

Dataset2 (Back-Court Stress Test)

This dataset is our own footage, recorded from a single fixed camera placed behind the court at a low amateur angle, which is the kind of setup a typical player or coach would actually use. The data is stored as raw .mp4 video files together with manual labels mapped to frame numbers. Unlike the first dataset, the frames are not pre-extracted. Our pipeline opens each video, reads the frames one by one, resizes them to a fixed resolution of 640x360 pixels, and rescales the labels to match. To keep the on-the-fly video decoding fast, we cap the workload at two clips and at most 450 frames per clip, giving a total of 672 frames. This cap is practical choice to limit processing time and not a limit of how much footage is available.

An important point is that both datasets are “frozen” before evaluation, meaning the exact set of frames is fixed in advance. Every tracking method is then run on this identical set of frames within each dataset. This guarantees that any difference in results comes from the methods themselves and not from differences in the input data.

B. Tracking Methods

We evaluate five tracking methods in total: one state-of-the-art deep learning network and four fundamental computer vision baselines. The four baselines are chosen to cover the two classical ideas behind tracking, namely appearance (following the ball by its color) and motion (following the ball by how pixels move).

1. TrackNet Model II(State-of-the-Art)

TrackNet is a deep learning network designed for tracking small, fast objects [1]. We use Model II, the version that takes three consecutive frames as input so that it can learn the ball’s motion pattern and not only its appearance. The network has about 10.7 million parameters and is built

from a VGG-16 style encoder followed by an upsampling decoder. Instead of directly predicting the coordinate, it outputs a heatmap in which a bright spot marks the most likely ball location. To read the final position from this heatmap, we apply a Hough circle detection step; if exactly one circle is found, its center is taken as the predicted position, otherwise the frame is reported as having no ball. We load the network with pre-trained weights and run it in inference mode only.

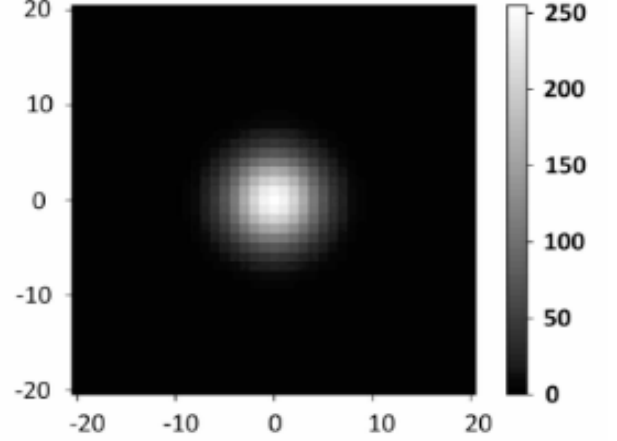


Figure 5: An example Heatmap produced by TrackNet next to the corresponding input frame, showing the bright spot on the ball

2. HSV Color segmentation (Appearance Detector)

This is the simplest baseline. For each frame independently, we convert the image to the HSV color space and keep only the pixels that fall inside a tennis-ball yellow range (lower bound [25, 50, 50], upper bound [55, 255, 255]). We then look for the most ball-like blob in this mask and take its center as the prediction. Because this method looks at each frame on its own and has no memory, it is a pure detector rather than a tracker.

3. CAMShift (Appearance Tracker)

CAMShift improves on simpler color segmentation by adding memory. It is started from the ball’s known position in the first confident frame, where it builds a color histogram model of the ball. In the following frames, it back-projects this color model and shifts a search window toward the area that best matches the ball’s color, while adapting window size. If the window collapses, grows too large, or moves off the frame, the track is considered lost and is re-started from the ground truth when the ball reappears. We use a search window of 20 pixels around the seed position

4. Lucas Kanade Optical Flow(Motion Tracker)

This method follows the ball using motion instead of color. It is based on the brightness-constancy assumption, which states that a small image patch keeps the same brightness as it moves between frames. We first place a window around the ball’s known starting position and detect strong corner features inside it. These features are then tracked from frame to frame using the pyramidal Lucas-Kanade method, and the average position of the surviving features is taken as the ball’s predicted location.

We use a tracking window of 15x15 pixels with two pyramid levels.

5. MOG2 Background Subtraction with Kalman Filter(Kinematic Baseline)

This baseline combines motion detection with motion prediction. A MOG2 background subtractor learns which parts of the scene are static and highlights the moving parts, which should include the ball. The detected ball position is then fed into a Kalman filter, which models the ball's position and velocity and predicts where it will appear next. When the detector finds the ball, the filter is corrected with that measurement; when the detector fails, the filter coasts on its prediction for a short time. To keep this fair, we limit coasting to a maximum of five consecutive frames, so that long blind predictions do not pollute the results.

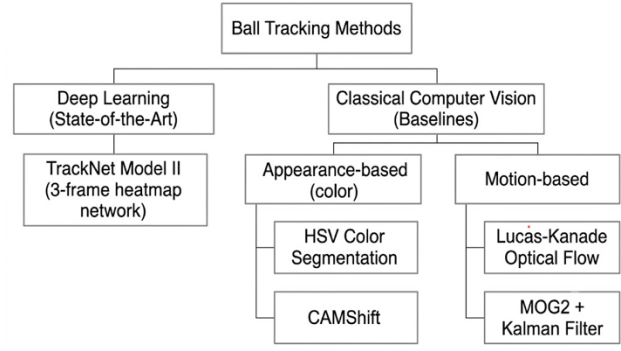


Figure 6: A simple diagram showing the five-methods

C. Implementation and Configuration

All methods are implemented in Python using OpenCV. TrackNet runs on a GPU(A100), while the four classical baselines run on the CPU. The four classical methods that need a starting point (CAMShift, Lucas-Kanade, and the Kalman Tracker after a loss) are initialized from the ground-truth ball position, so that we measure how well each method tracks once it is correctly started, rather than how well it finds the ball from scratch.

D. Evaluation Protocol

We compare the methods using localization accuracy, which measures how close each prediction is to the true ball position. For a frame where the ball is present and the method reports a detection, the positioning error (PE) is the Euclidean distance between the predicted position and the ground truth:

$$PE = \sqrt{(x_{gt} - x_{pred})^2 + (y_{gt} - y_{pred})^2} \quad (1)$$

Following the original TrackNet paper [1], a detection is counted as correct if its positioning error is 5 pixels or less. This threshold is based on the fact that a tennis ball in these videos is only about 5 pixels in radius, so an error within this range does not change the trajectory in any meaningful way. We report three numbers for each method: the mean positioning error over all its detections, the success rate (the percentage of detections within the 5-pixel threshold), and the number of frames in which the method reported a lock.

For TrackNet we also report classification metrics (precision, recall and F1-score), because the network naturally decides whether a ball is present or not. We deliberately do not report these classification metrics for the four classical baselines. The reason is that on cluttered frames these simple methods frequently lock onto the wrong object, such as court line, a logo, or a player, which produces a large number of false detections that would make precision and recall uninformative. Instead, we judge the classical methods purely on how accurately they localize the ball when they do report a lock. As a result, a very large mean positioning error for a classical method should be read as a sign of frequent false locks, while the success rate is more honest measure of useful performance.

Finally, we note that the two datasets contain different number of frames (815 and 672) and come from different scenes. For this reason, comparisons within a single dataset are exact and fair, since all methods see identical frames, while comparisons across the two datasets are treated as qualitative, showing how each method's behavior changes with the camera viewpoint rather than as a precisely matched head-to-head

III. EXPERIMENTAL RESULTS AND ANALYSIS

All experiments were conducted on Google Colab using a single NVIDIA A100 GPU with a high-RAM runtime configuration. TrackNet inference was run on the A100 GPU, while the four classical baselines were executed on the CPU. Running all five methods over both frozen datasets, together with the generation of the annotated verification videos, completed within a few minutes per dataset, since no training was performed and TrackNet was used in inference mode only with pre-trained weights. The loaded TrackNet Model II network contained approximately 10.7 million parameters. We first present the results on the broadcast control dataset, then on the back-court stress-test dataset, and finally we analyze the failure modes of each method.

A. Broadcast Control Results (Dataset 1)

Figure 8 summarizes the localization results for all five methods on the 815 broadcast frames. The difference between TrackNet and the classical baselines is very large. TrackNet locates the ball within 5 pixels in 97.81% of its detections, with a mean positioning error of only 4.76 pixels, and reaches a classification precision, recall, and F1-score of 97.81%, 95.83%, and 96.81% indicated in figure 7. These numbers are close to the performance reported in the original TrackNet paper [1] and confirm that the network work as intended on clean broadcast footage.

```

=====
[ SOTA DEEP LEARNING ] TRACKNET MODEL II VERIFICATION REPORT
=====
Total Evaluated Frames: 815
Classification Precision: 97.81%
Classification Recall: 95.83%
Classification F1-Score: 96.81%
Mean Positioning Error: 4.76 pixels
=====
  
```

Figure 7: TrackNet Model II Verification Report Dataset 1

DATASET 1 (CONTROL / BROADCAST VIEW) -- LOCALIZATION SUMMARY PE thr = 5px			
Method	Locks	Mean PE (px)	Success@thr
TrackNet Model II (SoTA)	776	4.76	97.81%
MOG2 + Kalman (Kinematic)	680	172.07	5.29%
HSV Segmentation (Appearance)	685	393.09	0.00%
CAMShift (Appearance Tracker)	438	77.77	63.01%
Lucas-Kanade (Motion Tracker)	682	196.31	1.91%

Figure 8: Localization Results on Dataset 1

The four classical baselines perform far worse. The best of them, CAMShift, reaches a success rate of 63.01%, while the others fall well below this: MOG2 + Kalman at 5.29%, Lucas-Kanade at 1.91%, and HSV color segmentation at 0.00%. The very large mean positioning errors of the weak methods (393 pixels for HSV, 196 for Lucas-Kanade, 172 for MOG2 + Kalman) do not mean the ball is “almost” found and slightly mislocated. Instead, as explained in the methodology, they show that these methods frequently lock onto the wrong object somewhere else on the court, so their detections are spread far from the true ball position. CAMShift does better than the rest because the ball’s yellow color stands out clearly against the blue court, which gives its color model something stable to follow.

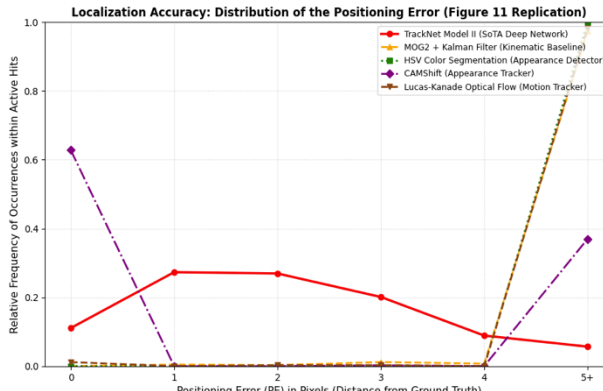


Figure 9: The five-way positioning-error distribution plot generated by the notebook for Dataset1. This is the report’s equivalent of Figure 11 in the TrackNet paper.

B. Back-Court Stress-Test Results (Dataset 2)

Figure 11 shows the results on the 672 back-court frames recorded from a single fixed camera. Here the picture changes in an interesting way, and this is the central finding of our study. TrackNet locates the ball within 5 pixels in 37.86% of its detections, with a mean positioning error of 15.42 pixels, and reaches a classification precision, recall, and F1-score of 37.86%, 92.07%, and 53.66% indicated in figure 10.

[SOTA DEEP LEARNING] TRACKNET MODEL II VERIFICATION REPORT	
Total Evaluated Frames:	672
Classification Precision:	37.86%
Classification Recall:	92.07%
Classification F1-Score:	53.66%
Mean Positioning Error:	15.42 pixels

Figure 10: TrackNet Model II Verification Report-Dataset 2

DATASET 2 (STRESS-TEST / ADVERSARIAL VIEW) -- LOCALIZATION SUMMARY PE thr = 5px			
Method	Locks	Mean PE (px)	Success@thr
TrackNet Model II (SoTA)	246	15.42	84.96%
MOG2 + Kalman (Kinematic)	264	17.16	60.23%
HSV Segmentation (Appearance)	264	216.15	10.61%
CAMShift (Appearance Tracker)	196	19.28	43.37%
Lucas-Kanade (Motion Tracker)	264	66.13	1.52%

Figure 11: Localization Results on Dataset 2

TrackNet still localizes well when it commits to a detection, reaching an 84.96% success rate and a mean positioning error of 15.42 pixels, both clearly better than any classical method on this dataset. However, its classification precision drops sharply to 37.86%, even though its recall stays high at 92.07%. This combination is important: the high recall means TrackNet almost always reacts when the ball is present, but the low precision means it also produces many false detections on a viewpoint it was not trained for. In other words, the network rarely misses the ball, but it is no longer confident about only firing on the ball.

At the same time, the MOG2 + Kalman baseline improves dramatically compared to its broadcast performance, rising from 5.29% success rate on Dataset 1 to 60.23% on Dataset 2. The reason is mechanical and easy to explain: a fixed camera produces a static background, which is exactly the condition that background subtraction is designed for. With the camera no longer moving, the moving ball stands out cleanly, and the Kalman filter can smooth and predict its motion effectively. CAMShift also remains unstable at 43.37%, while HSV (10.61%) and Lucas-Kanade (1.52%) remain weak.

The key observation is the reversal between the two datasets. On broadcast video, TrackNet dominates and the kinematic baseline is almost useless; on fixed-camera video, TrackNet becomes far less reliable at avoiding false detections, while the kinematic baseline becomes genuinely competitive. This shows that the strongest method is not the same in every situation, and that the right choice depends heavily on how the video was recorded.

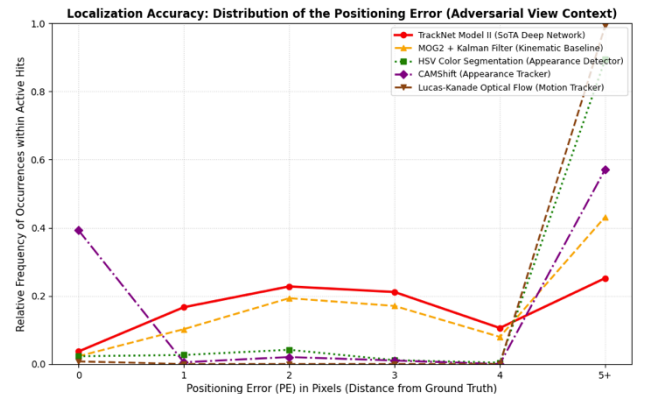


Figure 12: The five-way positioning-error distribution plot generated by the notebook for Dataset2. This is the report’s equivalent of Figure 11 in the TrackNet paper.

C. Qualitative Verification and Failure Analysis

To visually confirm the numerical results, both pipelines produce an annotated verification video in which the ground-truth position is drawn as a green square, the TrackNet prediction as a red circle, the classical prediction as an orange triangle, and a fading trajectory line traces the recent ball path.

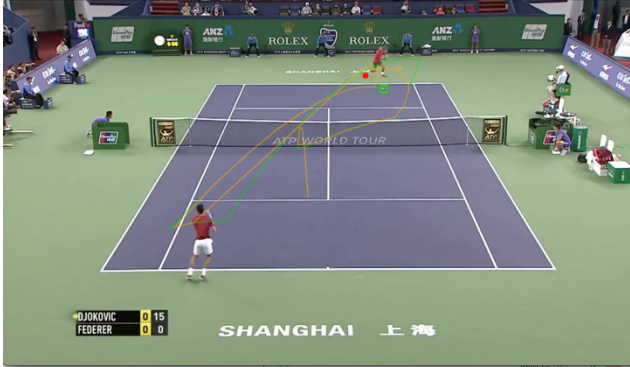


Figure 13: A screenshot from verification_dataset1.mp4

On the broadcast video, the verification frames show the red TrackNet marker staying tightly on the ball throughout the rally, while the orange classical trail spreads across the court and frequently jumps to the court lines and the players. This is the visual form of the false-lock problem behind the large mean errors in figure-8.



Figure 14: A screenshot from verification_dataset2.mp4

On the back-court video, the verification frames show TrackNet still finding the ball at distance, but they also reveal the false detections that lower its precision, where the red marker occasionally appears on background regions. They also show that the classical motion-based detection works more cleanly here than on broadcast footage, consistent with the improved kinematic results. From these results we can identify clear and explainable failure modes for each method. HSV color segmentation fails on broadcast video because many court elements share the ball's color, so it locks onto the wrong yellow region. Lucas-Kanade fails on both datasets, and most severely on fast motion, because the ball moves too far between frames for its small search window to follow, so its feature points are quickly lost or drift onto the background. This is the clearest demonstration of why a simple local motion model cannot keep up with a high-

speed object. CAMShift holds a lock only while the ball's color stays distinct and drifts onto the background once the ball is lost. MOG2 + Kalman depends almost entirely on the camera being still, which is why it is weak on broadcast footage and strong on fixed-camera footage. Most notably, TrackNet itself fails on the back-court viewpoint by producing many false positives, which is direct evidence that even a strong deep learning model is sensitive to a change in camera geometry away from its training conditions.

IV. CONCLUSION

In this work, we presented a comprehensive evaluation of the state-of-the-art TrackNet Model II against four fundamental computer vision trackers (HSV color segmentation, CAMShift, Lucas-Kanade optical flow, and a MOG2 background subtractor paired with a Kalman filter). Our systematic comparison across two distinct camera viewpoints demonstrated that the optimal tracking method is highly dependent on the viewing geometry and recording conditions.

On standard broadcast footage, the deep learning-based TrackNet clearly dominated, achieving a 97.81% success rate and a minimal positioning error of 4.76 pixels. In contrast, the classical appearance-and motion-based baselines failed significantly in this dynamic environment, frequently locking onto incorrect court elements or players, leading to a success rates below 64%.

However, testing on our fixed back-court footage revealed critical vulnerabilities in the deep learning model. When exposed to a viewpoint it was not trained on, TrackNet's classification precision sharply fell to 37.86% due to a high volume of false detections. Conversely, the static nature of the fixed camera allowed the kinematic MOG2 and Kalman filter baseline to thrive, dramatically improving its success rate to 60.23% by cleanly isolating the moving ball against a stable background.

Ultimately, these findings highlight that while TrackNet is the strongest overall tracker, it remains highly sensitive to domain shifts and camera geometry. For real-world applications (particularly amateur setups utilizing a single static camera) computationally inexpensive motion-based baselines can serve as highly competitive and reliable alternatives to complex deep neural networks.

REFERENCES

- [1] Y.-C. Huang, I. -N. Liao, C. -H. Chen, T.-U. Ik, and W. -C. Peng, "TrackNet: A Deep Learning Network for Tracking High-speed and Tiny Objects in Sport Applications," arXiv preprint arXiv:1907.03698, 2019.